Kruskal's Algorithm

Session No.:
Course Name: Analysis and Design of Algorithms
Course Code: R1UC407B
Instructor Name: Dr. Mili Dhar

Review of the key concepts of session no.

1. Kruskal's algorithm is used to find the **Minimum Spanning Tree (MST)** of a connected/disconnected, undirected, weighted graph.
2. Sort all edges by weight and add the smallest edge that doesn't form a cycle.
3. If there are multiple edges with the same weight, any of them can be chosen without affecting the overall MST cost.
4. Different choices for these equally weighted edges can result in different MSTs.
5. It is particularly useful for sparse graphs where the number of edges is much less than the maximum possible number of edges.

If you were designing a network and wanted to connect all nodes with the minimum possible cost, what factors would you consider to make it efficient?

At the end of this session students will be able to

Learning Outcome:

- Analyse Kruskal's Algorithm in a weighted graph.

Session Outline

1. Pseudo-code of Kruskal's Algorithm

2. Dry-Run of pseudocode

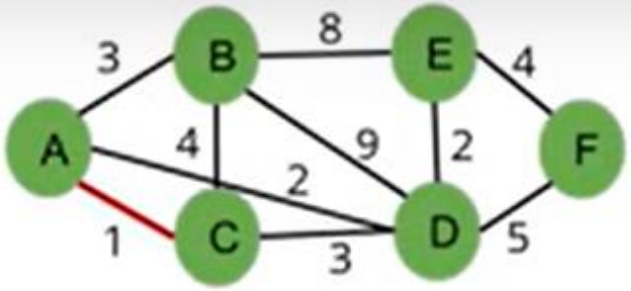
3. Analyse Time and space complexity

Pseudo-code of Kruskal's Algorithm

```
MST-KRUSKAL(G, w)
A = ∅
for each vertex v ∈ G.V
    MAKE-SET(v)
sort the edges of G.E into nondecreasing order by weight w
for each edge (u, v) ∈ G.E, taken in nondecreasing order by weight
    if FIND-SET(u) ≠ FIND-SET(v)
        A = A ∪ {(u, v)}
        UNION(u, v)
return A
```

Learning Activity- 1 (Dry-run of Pseudo-code)

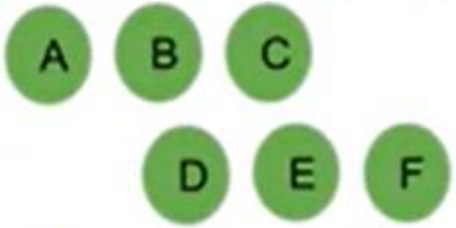
Example



MST-KRUSKAL's ALGORITHM(G, w) {

1. A = ϕ
2. for each vertex v ε V[G]
3. do MAKE-SET(v)
4. Sort the edges of E into non-decreasing order by weight w.
5. for each edge (u, v) ε E[G] taken in non-decreasing order by weight.
6. do if FIND-SET(u) ≠ FIND-SET(v)
7. then A = A U { (u, v) }
8. UNION(u,v)
9. return A

A = { }

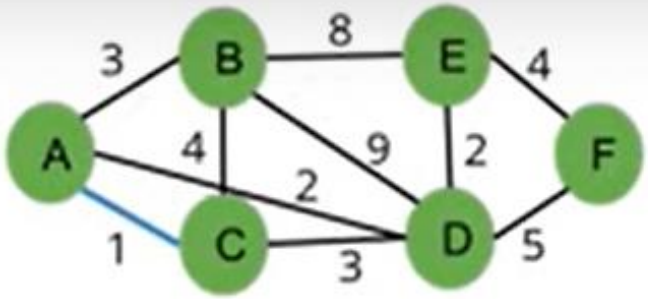


FIND-SET(A) ≠ FIND-SET(C)
A ≠ C

Edges	Weight
A-C	1
A-D	2
E-D	2
A-B	3
C-D	3
B-C	4
E-F	4
D-F	5
B-E	8
B-D	9

Learning Activity- 1 (Dry-run of Pseudo-code)

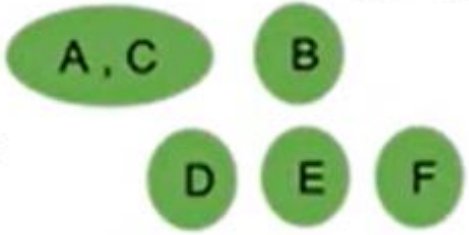
Example



MST-KRUSKAL's ALGORITHM(G, w) {

1. A = ϕ
2. for each vertex v ε V[G]
3. do MAKE-SET(v)
4. Sort the edges of E into non-decreasing order by weight w.
5. for each edge (u, v) ε E[G] taken in non-decreasing order by weight.
6. do if FIND-SET(u) ≠ FIND-SET(v)
7. then A = A U { (u, v) }
8. UNION(u,v)
9. return A

A = { A, C }

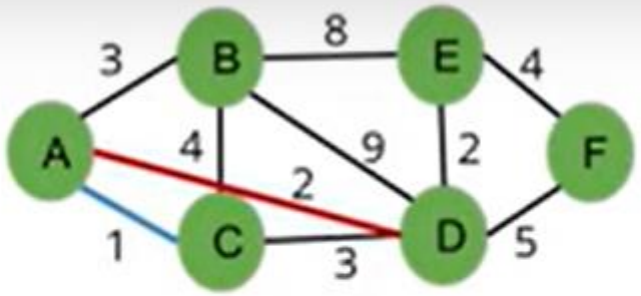


FIND-SET(A) ≠ FIND-SET(C)
A ≠ C

Edges	Weight
A-C	1
A-D	2
E-D	2
A-B	3
C-D	3
B-C	4
E-F	4
D-F	5
B-E	8
B-D	9

Learning Activity- 1 (Dry-run of Pseudo-code)

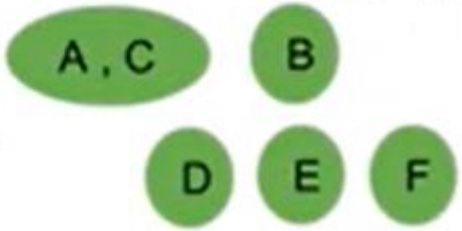
Example



MST-KRUSKAL's ALGORITHM(G, w) {

1. A = ϕ
2. for each vertex v ε V[G]
3. do MAKE-SET(v)
4. Sort the edges of E into non-decreasing order by weight w.
5. for each edge (u, v) ε E[G] taken in non-decreasing order by weight
6. do if FIND-SET(u) ≠ FIND-SET(v)
7. then A = A U { (u, v) }
8. UNION(u,v)
9. return A

A = { A, C }

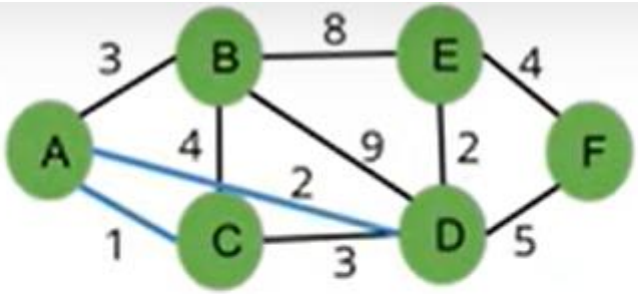


FIND-SET(A) ≠ FIND-SET(D)
A ≠ D

Edges	Weight
A-C	1
A-D	2
E-D	2
A-B	3
C-D	3
B-C	4
E-F	4
D-F	5
B-E	8
B-D	9

Learning Activity- 1 (Dry-run of Pseudo-code)

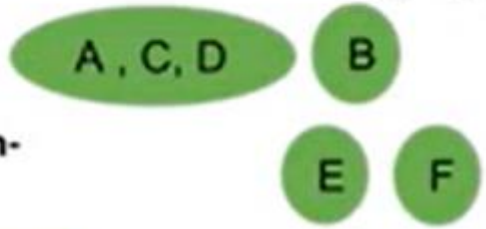
Example



MST-KRUSKAL's ALGORITHM(G, w) {

1. A = ϕ
2. for each vertex v ε V[G]
3. do MAKE-SET(v)
4. Sort the edges of E into non-decreasing order by weight w.
5. for each edge (u, v) ε E[G] taken in non-decreasing order by weight.
6. do if FIND-SET(u) ≠ FIND-SET(v)
7. then A = A U { (u, v) }
8. UNION(u,v)
9. return A

A = { A, C, D }



FIND-SET(A) ≠ FIND-SET(D)
A ≠ D

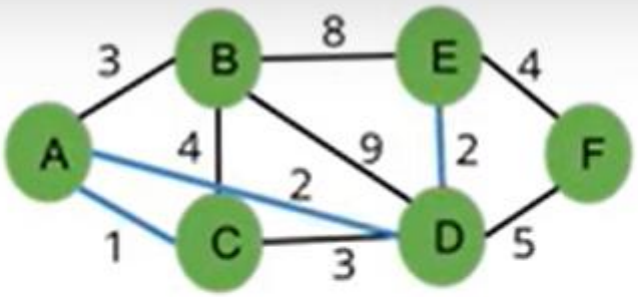
Edges	Weight
A-C	1
A-D	2
E-D	2
A-B	3
C-D	3
B-C	4
E-F	4
D-F	5
B-E	8
B-D	9

Learning Activity- 1 (Dry-run of Pseudo-code)

Example

MST-KRUSKAL's ALGORITHM(G, w) {

1. A = ϕ
2. for each vertex v ε V[G]
3. do MAKE-SET(v)
4. Sort the edges of E into non-decreasing order by weight w.
5. for each edge (u, v) ε E[G] taken in non-decreasing order by weight.
6. do if FIND-SET(u) ≠ FIND-SET(v)
7. then A = A U { (u, v) }
8. UNION(u,v)
9. return A



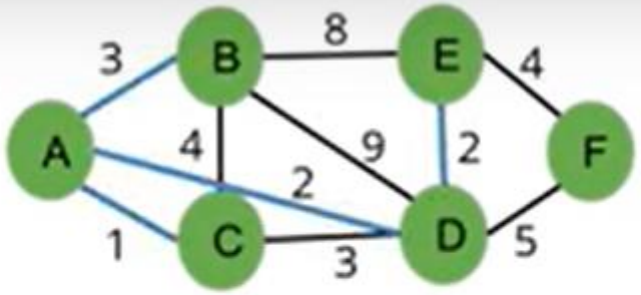
A = { A, C, D, E}

FIND-SET(E) ≠ FIND-SET(D)
E ≠ A

Edges	Weight
A-C	1
A-D	2
E-D	2
A-B	3
C-D	3
B-C	4
E-F	4
D-F	5
B-E	8
B-D	9

Learning Activity- 1 (Dry-run of Pseudo-code)

Example



MST-KRUSKAL's ALGORITHM(G, w) {

1. A = ϕ
2. for each vertex v ε V[G]
3. do MAKE-SET(v)
4. Sort the edges of E into non-decreasing order by weight w.
5. for each edge (u, v) ε E[G] taken in non-decreasing order by weight.
6. do if FIND-SET(u) ≠ FIND-SET(v)
7. then A = A U { (u, v) }
8. UNION(u,v)
9. return A

A = { A, B, C, D, E}

A , B, C, D, E

F

FIND-SET(A) ≠ FIND-SET(B)
A ≠ B

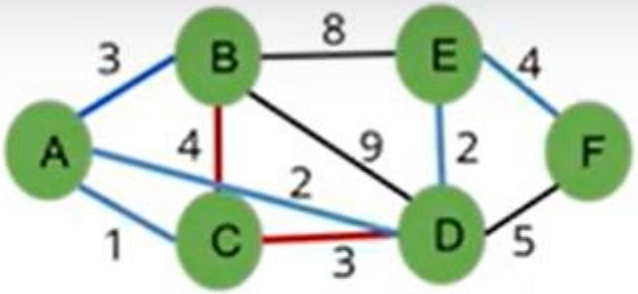
Edges	Weight
A-C	1
A-D	2
E-D	2
A-B	3
C-D	3
B-C	4
E-F	4
D-F	5
B-E	8
B-D	9

Learning Activity- 1 (Dry-run of Pseudo-code)

Example

MST-KRUSKAL's ALGORITHM(G, w) {

1. A = ϕ
2. for each vertex v ε V[G]
3. do MAKE-SET(v)
4. Sort the edges of E into non-decreasing order by weight w.
5. for each edge (u, v) ε E[G] taken in non-decreasing order by weight.
6. do if FIND-SET(u) ≠ FIND-SET(v)
7. then A = A U { (u, v) }
8. UNION(u,v)
9. return A



A = { A, B, C, D, E, F}

A , B, C, D, E, F

FIND-SET(E) ≠ FIND-SET(F)
A ≠ F

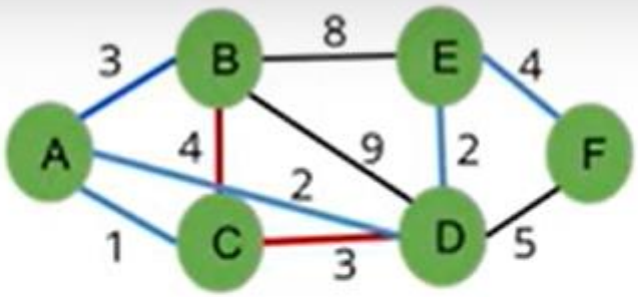
Edges	Weight
A-C	1
A-D	2
E-D	2
A-B	3
C-D	3
B-C	4
E-F	4
D-F	5
B-E	8
B-D	9

Learning Activity- 1 (Dry-run of Pseudo-code)

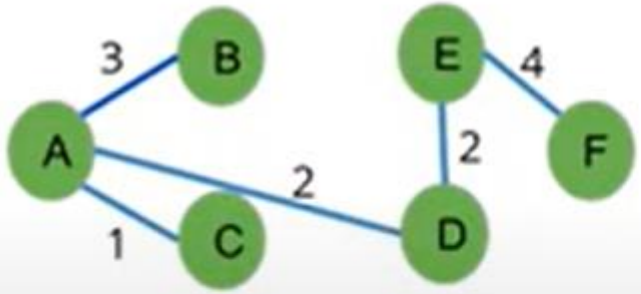
Example

MST-KRUSKAL's ALGORITHM(G, w) {

1. A = ϕ
2. for each vertex v ε V[G]
3. do MAKE-SET(v)
4. Sort the edges of E into non-decreasing order by weight w.
5. for each edge (u, v) ε E[G] taken in non-decreasing order by weight.
6. do if FIND-SET(u) ≠ FIND-SET(v)
7. then A = A U { (u, v) }
8. UNION(u,v)
9. return A



A = { A, C, D, E}



Wt = 1 + 2 + 2 + 3 + 4 = 12

Edges	Weight
A-C	1
A-D	2
E-D	2
A-B	3
C-D	3
B-C	4
E-F	4
D-F	5
B-E	8
B-D	9

Learning Activity-2

```
MST-KRUSKAL(G, w)
A = ∅                                                          O(1)
for each vertex v ∈ G.V
    MAKE-SET(v)                                                O(v)
sort the edges of G.E into nondecreasing order by weight w     O(ElogE)
for each edge (u, v) ∈ G.E, taken in nondecreasing order by weight
    if FIND-SET(u) ≠ FIND-SET(v)
        A = A ∪ {(u, v)}                                       O(ElogV)
        UNION(u, v)
return A
```

Total time: $O(E\log E) + O(E \log V)$
$= O(E\log V)$ or $O(E\log E)$

Conclusion

1. Go to wooclap.com
2. Enter the event code in the top banner

Event code
YANACF

Summary

1. Find the Minimum Spanning Tree by sorting edges and selecting the smallest edge that doesn't form a cycle.
2. Uses Union-Find data structure for cycle detection.
3. Time Complexity Analysis:
   - Sorting Edges: O(ElogE)
   - Union-Find Operations: O(ElogV)
   - Overall Time Complexity: O(ElogE) + O(ElogV)

4. Space Complexity Analysis:
   - Edge List Storage: O(E)
   - Union-Find Arrays: O(V)
   - Overall Space Complexity: O(E+V).

Next Session....

We will learn about another greedy approach